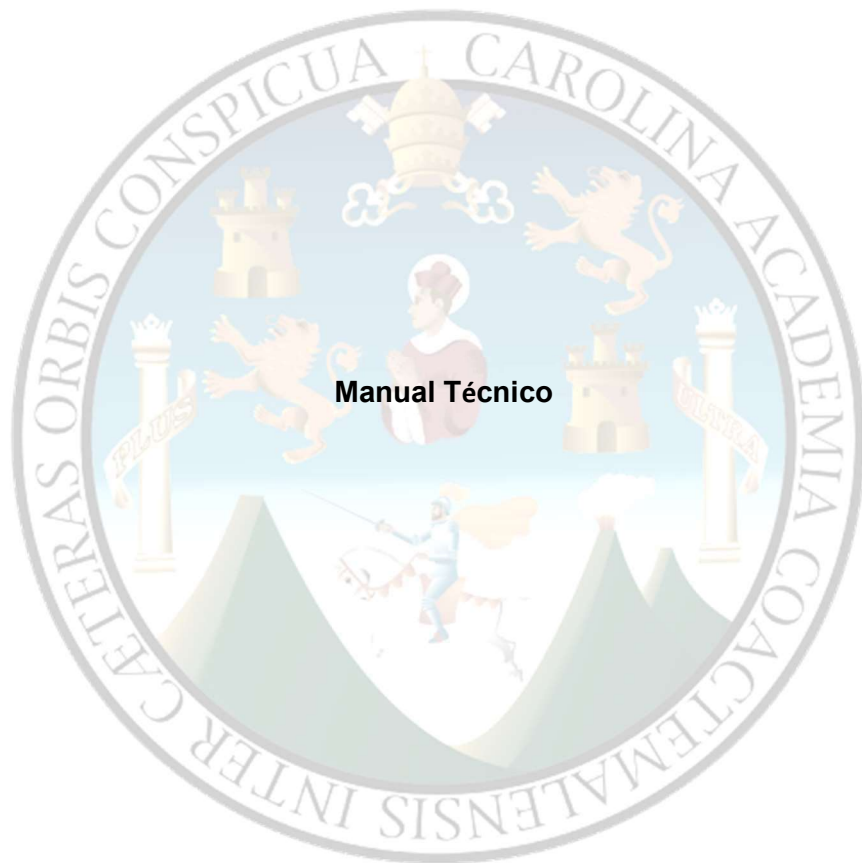


UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
SEGUNDO SEMESTRE
2025
ESTRUCTURAS DE DATOS



Nombre

Yury Urbano Santos Chagil

Carné

202113318

Manual Técnico

El presente Manual Técnico constituye un documento de referencia formal para el sistema de gestión de correos desarrollado en Object Pascal utilizando el entorno de desarrollo Lazarus/FPC. Su elaboración responde a la necesidad de contar con una guía estructurada que describa en detalle la arquitectura interna del proyecto, así como los procedimientos y funciones implementados en cada módulo.

El sistema integra el paradigma de la programación orientada a eventos con el uso de estructuras de datos dinámicas, tales como listas doblemente enlazadas, pilas, colas, listas circulares, listas de listas y una matriz dispersa, con el fin de gestionar de manera eficiente usuarios, contactos, correos, papelera, correos programados y comunidades.

Este manual está dirigido a docentes, estudiantes y desarrolladores que requieran comprender el funcionamiento interno de la aplicación, ofreciendo un análisis exhaustivo de las unidades (.pas), formularios (.lfm), procedimientos y funciones. Su propósito es servir como herramienta para la mantenibilidad, escalabilidad y evaluación académica del sistema, facilitando tanto el proceso de revisión como su futura extensión o mejora.

Teoría de las Estructuras de Datos Utilizadas – Fase 2

1) Árbol B (orden 5) – Favoritos

Definición

Un **Árbol B** es una estructura de búsqueda equilibrada multi-rama (m-aria) diseñada para minimizar la altura del árbol y optimizar accesos por bloque (común en discos/bases de datos). Cada nodo almacena **múltiples claves** y puede tener **múltiples hijos**.

Características

- **Orden 5** en el proyecto
 - hasta **4 claves** y **5 hijos** por nodo ($MAX_KEYS=4$).
- **Balanceado por diseño**: todas las hojas al mismo nivel; inserciones/eliminaciones redistribuyen/parten/unen nodos.
- **Complejidad**: búsquedas/altas/bajas en $O(\log_m n)$ con baja altura (pocos niveles).
- **Adecuado** cuando hay muchas claves y se prioriza “poca altura” del árbol.

Aplicación en el proyecto

- **Qué guarda**: correos marcados como **favoritos** del usuario.
 - Record TFavorito: id, remitente, estado, asunto, fecha, mensaje.
- **Dónde está**: btree_favoritos.pas.
Variable global: FavoritosBTree: PBNode (en app_state.pas).
- **Operaciones clave**:
BFav_Insert, BFav_Search, BFav_Delete, BFav_InOrder, BFav_ToDOT.
- **UI relacionada**:
 - form_bandeja.pas: botón **Favorito** (toggle) sincroniza B-Tree.
 - form_favoritos.pas: lista, ver por ID, eliminar, contador.
 - **Reporte**: GenerarReporteFavoritosBTreePorEmail → favoritos_btree.png.

2) Árbol BST – Contactos

Definición

Un **BST** (Binary Search Tree) es un árbol binario donde, para cada nodo, claves menores van al **subárbol izquierdo** y mayores al **derecho**.

Características

- **Ordenado** por clave (aquí el **email** normalizado).
- **Complejidad**: en promedio $O(\log n)$; peor caso $O(n)$ si se desbalancea (aceptable por tamaño típico).
- Fácil de **recorrer in-order** para listar ordenado.

Aplicación en el proyecto

- **Qué guarda:** contactos del usuario (para búsquedas/visualización/eliminación).
 - Record TBSTContacto: email, nombre, telefono.
- **Dónde está:** bst_contactos.pas.
Variable global: ContactosBST: PBST.
- **Operaciones:** BST_Init, BST_Insert, BST_Search, BST_Delete, BST_ToDOT.
- **Integraciones:**
 - form_agregar_contacto.pas: al agregar, inserta en la lista circular y (opcionalmente) refleja en BST.
 - form_contactos.pas: eliminar contacto sincroniza **lista y BST**.
 - **Reporte:** GenerarReporteContactosBSTPorEmail
 - contactos_bst.png.

3) Árbol BST – Comunidades (con lista simple de mensajes)

Definición

Otro **BST**, esta vez indexado por **nombre de comunidad** (normalizado). Cada nodo representa una comunidad y enlaza una **lista simple** de mensajes publicados.

Características

- Ordenado por **nombre de grupo** (clave).
- **Cada nodo** almacena metadatos de la comunidad y un **puntero a la lista** de mensajes.
- **Complejidad** de BST: promedio **$O(\log n)$** ; peor **$O(n)$** (número de comunidades suele ser moderado).
- Los mensajes se gestionan con **lista simple** (inserción $O(1)$ al frente).

Aplicación en el proyecto

- **Qué guarda:**
 - En el nodo BST (TBSTC):
nombre, fechaCreacion, msgsHead: PMsg, msgCount, L, R.
 - En la lista de mensajes (TMsg): correo, mensaje, fecha, sig.
- **Dónde está:** bst_comunidades.pas.
Variable global: ComunidadesBST: PBSTC (en app_state.pas).
- **Operaciones:**
 - BST: BSTC_Init, BSTC_Find, BSTC_Insert, BSTC_EnsureCommunity
 - Mensajes: BSTC_AddMessage, BSTC_ListMessages, BSTC_ListCommunities, BSTC_ToDOT.
- **Flujos/UI:**
 - **Root** crea comunidades (o se aseguran con BSTC_EnsureCommunity cuando se usan).
 - **Usuario estándar** publica con form_mensaje_comunidad.pas (solo si pertenece).
 - **Root** visualiza todo con FormMensajesComunidadesRoot.pas (memo con dump in-order).
 - **Reporte:** GenerarReporteComunidadesBST → comunidades_bst.png (muestra **Nombre, Fecha creación, #Mensajes**).

4) Lista simple – Mensajes por comunidad

Definición

Estructura lineal donde cada elemento apunta al siguiente. Aquí se usa por comunidad para **encadenar publicaciones**.

Características

- **Inserción $O(1)$** al frente (msgsHead).
- Recorrido **$O(k)$** para listar los **k** mensajes de un grupo.
- Muy simple y suficiente para el tamaño típico.

Aplicación en el proyecto

- **Record** TMsg: correo, mensaje, fecha, sig.
- **Campo en BST de comunidades:** msgsHead: PMsg, msgCount: Integer.
- **Operaciones:** creación de nodo mensaje, encadenado frontal en BSTC_AddMessage, recorridos en BSTC_ListMessages y en el visor root.

Estructura del Proyecto (Unidades y Estructuras)

Núcleo y Estado Global

main.pas

- **Propósito:** Pantalla de inicio de sesión (login) e inicialización del sistema.
- **Form principal:** TForm1 (campos Email/Password y botones Login/Registrar).
- **Variables globales (históricas):**
 - UsuarioActualEmail: String; (*→ ahora centralizada en app_state.pas, ideal referenciarla desde allí*).
- **Inicializa:**
 - Usuarios, Bandejas, Papelera, Cola de Programados, Contactos, Comunidades.
- **Lógica:** Si el email es root@edd.com abre menú root; si no, menú usuario.

app_state.pas

- **Propósito:** Variables de estado global accesibles desde todas las pantallas.
- **Variables globales típicas:**
 - UsuarioActualEmail: String;
 - FavoritosBTree: PBNODE; (*Árbol B de favoritos*)
 - ComunidadesBST: PBSTC; (*BST de comunidades*)
 - (*si usas centralización extra, aquí puedes declarar otras referencias globales*)

Usuarios y Contactos

usuarios.pas

- **Propósito:** Lista simple de usuarios; carga/validación desde JSON.
- **Records / Tipos:**
 - PUsuario = ^TUsuario;
 - TUsuario = record id, nombre, usuario, password, email, telefono; siguiente: PUsuario; end;
- **Globals:**
 - ListaUsuarios: PUsuario;
- **Funciones clave:**
 - InicializarUsuarios;
 - CargarUsuariosDesdeJSON(path); (*valida IDs y emails únicos*)
 - ExisteEmail(email), BuscarUsuarioPorEmail(email [,password]);
 - AgregarUsuario(...);

contactos.pas

- **Propósito:** Lista **circular** de contactos por dueño (owner).
- **Records / Tipos:**
 - PContacto = ^TContacto;
 - TContacto = record id, ownerEmail, nombre, email, telefono; siguiente: PContacto; end;
 - TListaContactos = record cabeza: PContacto; end;
- **Globals:**
 - ListaContactos: TListaContactos;
- **Funciones clave:**
 - InicializarContactos(L);
 - AgregarContacto(L, id, owner, nombre, email, tel);
 - ExisteContactoEmail(L, owner, email): Boolean;
 - EsContacto(L, owner, email): Boolean;
 - GuardarContactosEnJSON / CargarContactosDesdeJSON;
 - ObtenerSiguienteIDContacto(L): Integer;

bst_contactos.pas

- **Propósito:** BST de contactos para búsquedas/visualización y reporte.
- **Records / Tipos:**
 - TBSTContacto = record email, nombre, telefono; end;
 - PBST = ^TBST;
 - TBST = record key: String; data: TBSTContacto; L, R: PBST; end;
- **Globals:**
 - ContactosBST: PBST;
- **Funciones clave:**
 - BST_Init(var T); BST_Insert(var T; const C);
 - BST_Search(T, email): PBST;
 - BST_Delete(var T, email): Boolean;
 - BST_ToDOT(T, fileDot);

Correos: Bandeja, Papelera y Programados

lista_doble.pas (*bandeja de correos*)

- **Propósito:** Lista doblemente enlazada de correos de cada usuario.
- **Records / Tipos:**
 - PCorreo = ^TCorreo;
 - TCorreo = record
 - id: Integer; remitente: String; estado: String; programado: Boolean;
 - asunto, fecha, mensaje: String; favorito: Boolean;
 - anterior, siguiente: PCorreo;
 - end;
 - TBandeja = record cabeza, cola: PCorreo; tamano: Integer; end;
 - PBandeja = ^TBandeja;
- **Funciones clave:**
 - InicializarBandeja(var B); InsertarCorreo(var B; ...);
 - BuscarCorreo(B, id): PCorreo; EliminarCorreo(var B, id): Boolean;
 - OrdenarPorAsunto(var B);

bandejas.pas

- **Propósito:** Mapa/gestor de bandejas por email.
- **Funciones clave:**
 - InicializarBandejas;
 - ObtenerBandejaPtr(ownerEmail): PBandeja;
 - EntregarCorreoA(para, de, asunto, fecha, msg, id, estado, programado);

pila_papelera.pas

- **Propósito:** Pila (LIFO) de correos eliminados.
- **Records / Tipos:**
 - TCorreoInfo = record remitente, estado, asunto, fecha, mensaje: String; id: Integer; end;
 - PPilaNodo = ^TPilaNodo; TPilaNodo = record dato: TCorreoInfo; siguiente: PPilaNodo; end;
 - TPila = record tope: PPilaNodo; end;
- **Globals:** PapeleraGlobal: TPila;
- **Funciones clave:** InicializarPapelera(var P); PushCorreo(var P, info); PopCorreo(var P, out info);

cola_correos.pas

- **Propósito:** Cola (FIFO) de correos programados.
- **Records / Tipos:** TCorreoProgramado = record remitente, destinatario, asunto, fecha, hora: String; end;
 - Nodos de cola PColaNodo y TCola.
- **Globals:** ColaGlobal: TCola;
- **Funciones clave:** InicializarCola(var C); Encolar(var C, dato); Desencolar(var C, out dato);

Borradores (AVL) y Favoritos (Árbol B)

avl_borradores.pas

- **Propósito:** AVL para borradores.
- **Records / Tipos:**
 - TBorrador = record id: Integer; remitente, destinatario, asunto, mensaje: String; end;
 - PAVL_Borr = ^TAVL_Borr;
 - TAVL_Borr = record data: TBorrador; h: Integer; L, R: PAVL_Borr; end;
- **Globals:** BorradoresAVL: PAVL_Borr;
- **Funciones clave:**
 - BAVL_Insert(var T, Borrador); BAVL_Find(T, id): PAVL_Borr;
 - BAVL_Delete(var T, id): Boolean;
 - BAVL_ToStrings_PreOrder/InOrder/PostOrder(T, lines);
 - BAVL_ToDOT(T, fileDot);
 - NextDraftId: Integer; (*helper para IDs*)

btree_favoritos.pas

- **Propósito:** Árbol B (orden 5) de **favoritos**.
- **Constantes:** MAX_KEYS = 4; MIN_KEYS = 2;
- **Records / Tipos:**
 - TFavorito = record id: Integer; remitente, estado, asunto, fecha, mensaje: String; end;
 - PBNode = ^TBNode; TBNode = record
 - n: Integer;
 - key: array[1..MAX_KEYS] of Integer;
 - data: array[1..MAX_KEYS] of TFavorito;
 - child: array[0..MAX_KEYS] of PBNode;
 - leaf: Boolean;
 - end;
- **Funciones clave:**
 - BFav_Init(var T);
 - BFav_Search(T, id): PBNode;
 - BFav_Insert(var T, F: TFavorito);
 - BFav_Delete(var T, id): Boolean;
 - BFav_InOrder(T, proc: TFavProc);
 - BFav_ToDOT(T, fileDot);

Comunidades (Fase 1 – listas / Fase 2 – BST con mensajes)

comunidades.pas (Fase 1: listas simples)

- **Records / Tipos:**
 - Miembros: PMiembro = ^TMiembro; TMiembro = record email: string; sig: PMiembro; end;
 - TListaMiembros = record cabeza: PMiembro; tam: Integer; end;
 - Comunidades: PComunidad = ^TComunidad;
 - TComunidad = record nombre: string; miembros: TListaMiembros; sig: PComunidad; end;
 - TListaComunidades = record cabeza: PComunidad; tam: Integer; end;
- **Globals:** ListaComunidades: TListaComunidades;
- **Funciones clave:**
 - InicializarComunidades(L);
 - CrearComunidad(nombre): Boolean;
 - BuscarComunidad(nombre): PComunidad;
 - AgregarMiembro(comunidad, email): Integer; *(valida existencia y duplicados)*
 - ListarComunidades(L, items); ListarMiembros(comunidad, items);
 - ListarComunidadesDeUsuario(ownerEmail, items); *(para Fase 2, si lo agregaste)*

bst_comunidades.pas (Fase 2: BST + mensajes)

- **Records / Tipos:**
 - Mensajes: PMsg = ^TMsg; TMsg = record correo, mensaje, fecha: String; sig: PMsg; end;
 - BST: PBSTC = ^TBSTC;
 - TBSTC = record
 - key, nombre, fechaCreacion: String;
 - msgsHead: PMsg; msgCount: Integer;
 - L, R: PBSTC;
 - end;
- **Globals:** ComunidadesBST: PBSTC; (*en app_state.pas*)
- **Funciones clave:**
 - BSTC_Init(var T);
 - BSTC_Find(T, nombreGrupo): PBSTC;
 - BSTC_Insert(var T, nombreGrupo, fechaCreacion): PBSTC;
 - BSTC_EnsureCommunity(var T, nombreGrupo, fechaCreacion): PBSTC;
 - BSTC_AddMessage(var T, nombreGrupo, correo, mensaje, fechaPub): Boolean;
 - BSTC_ListMessages(T, nombreGrupo, out SL: TStringList): Boolean;
 - BSTC_ListCommunities(T, items);
 - BSTC_ToDOT(T, fileDot);

Reportes y Carga Masiva

reportes_usuario.pas

- **Propósito:** Generación de reportes .dot/.png por usuario.
- **Funciones clave:**
 - GenerarReportesUsuarioPorEmail(email [, BaseDir]);
 - ReporteCorreosRecibidos(userEmail, carpeta);
 - ReportePapelera(userEmail, carpeta);
 - ReporteProgramados(userEmail, carpeta);
 - ReporteContactos(userEmail, carpeta);
 - GenerarReporteBorradoresAVLPorEmail(UsuarioEmail);
 - GenerarReporteContactosBSTPorEmail(UsuarioEmail);
 - GenerarReporteFavoritosBTreePorEmail(UsuarioEmail);
 - *Helpers:* RunDot(dotPath, pngPath), MkUserFolder(...), Esc(...).

carga_masiva_correos.pas

- **Propósito:** Cargar correos desde correos.json **directo a Papelera** (según consigna de prueba).
- **Funciones clave:**
 - CargarCorreosDesdeJSON_A_Papelera(path, out agregados, out rechazados, log: TStringList);
 - Normaliza campos, valida estructura y apila en PapeleraGlobal.

Formularios (UI)

form_root.pas (menú del root)

- **Botones típicos:** Carga masiva de usuarios / reportes globales / gestión comunidades (Fase 1) / Ver mensajes comunidades (Fase 2).
- **Acciones:** Llamadas a generadores de reportes, abrir FormComunidades (F1) y FormMensajesComunidadesRoot (F2).

form_usuario.pas (menú del usuario estándar)

- **Botones:** Bandeja, Enviar, Papelera, Programados, Borradores, Contactos, Perfil, Reportes, Favoritos, Mensaje Comunidad.
- **Acciones destacadas:**
 - Button9Click: genera **todos los reportes** a usuario-Reportes/.
 - BtnFavoritosClick: abre FormFavoritos.
 - BtnBorradoresClick: abre FormBorradores.
 - BtnContactosClick: abre FormContactos.
 - BtnProgramarClick, BtnPapeleraClick, etc.

form_bandeja.pas

- **Muestra:** lista de correos de la bandeja (ListCorreos) con estado, asunto, remitente, **(ID)** y marca de favorito ***.
- **Acciones:**
 - Ver (marca como leído), Eliminar (a papelera), Ordenar A–Z, Favorito (toggle y sincroniza B-Tree).
- **Métodos:** CargarBandejaPtr(PBandeja); TryGetIDSeleccionado(out id);

form_enviarcorreo.pas

- **Campos:** EditPara, EditAsunto, MemoMensaje, Botones *Enviar, Borrador, Cerrar*.
- **Reglas:** Solo envía a **contactos**; valida formato de email; guarda en borrador si se desea; al enviar desde borrador elimina del AVL.

form_borradores.pas

- **Funciones:** Recorridos Pre/In/Post; Cargar por ID → abre form_enviarcorreo con datos; Reporte Graphviz del AVL.

form_contactos.pas

- **Muestra/Navega:** contactos del usuario (lista circular) siguiente/anterior; eliminar contacto (remueve de BST y lista).
- **Labels:** LblNombre, LblEmail, LblTelefono, LblPos.

form_agregar_contacto.pas

- **Agrega** contacto por email (debe existir como usuario, no duplicado).
- **Guarda** en JSON si aplica.

form_correosprogramados.pas y form_programarcorreo.pas

- **Gestiona** la cola de programados (ver/crear).

form_papelera.pas

- **Visualiza** la pila de papelera (LIFO).

form_perfil.pas

- **Actualiza** datos de perfil del usuario (si aplica).

form_favoritos.pas

- **Pantalla de favoritos (Árbol B):**
 - Listar, contador, Ver por ID (marca leído y re-sincroniza), Eliminar por ID (desmarca en bandeja).
 - **Refresca** la bandeja si está abierta.
 - Genera reporte favoritos_btree.png (vía botón de reportes en form_usuario.pas).

form_comunidades.pas

- **Root:** crear comunidad y agregar miembros (validaciones de existencia/duplicados).
- **Combo** con comunidades, memo log de acciones.

form_comunidades_bst.pas

- **Root:** crear **comunidad principal** en BST
(Si aprovechaste el alta al publicar, esta form puede ser opcional.)

form_mensaje_comunidad.pas

- **Usuario estándar:** publicar mensaje a comunidades donde es miembro.
- **Controles:** ComboComunidades, EditMensaje, **Publicar** (añade a BST de comunidades), **Cerrar**.

FormMensajesComunidadesRoot.pas

- **Root:** ver **todas** las comunidades y **todos** los mensajes.
- **Controles:** TMemo MemoTodo, **Refrescar**, **Cerrar**.
- **Render:** in-order del BST y listado de cada lista de mensajes.

Flujos funcionales clave

Login

- main.pas: verifica usuario/clave (usuarios.pas).
- Redirige a form_root (root) o form_usuario (usuario).
- Setea UsuarioActualEmail (en app_state.pas).

Enviar correo (redactar/envío)

- form_enviarcorreo.pas:
 - Valida **formato** de email y que **destinatario es contacto** (EsContacto).
 - Inserta en bandeja del destinatario (EntregarCorreoA).
 - Si el correo viene de un **borrador** (cargado con DraftIdLoaded), elimina del AVL tras enviar.

Borradores

- Desde enviar correo: botón **Borrador** guarda en AVL (BAVL_Insert).
- form_borradores: recorridos (pre/in/post), **cargar por ID** → abre enviar correo, **auto-delete** al enviar.
- Reporte: borradores_avl.dot/png.

Bandeja

- Visualizar correos, no leídos, ver correo (marca L), ordenar por asunto.
- **Favorito**: toggle → sincroniza **Árbol B**; reflecta en listado (**).
- Eliminar → empuja a **papelera**; si era favorito, elimina también del B-Tree.

Favoritos (Árbol B)

- form_favoritos: listar, ver por ID (sincroniza “leído”), eliminar por ID (y desmarca en bandeja).
- Reporte: favoritos_btree.dot/png.

Contactos (BST)

- Agregar contacto (valida que el usuario exista en el sistema y no duplicado).
- form_contactos: navegar (circular), eliminar sincronizado en lista y BST.
- Reporte: contactos_bst.dot/png.

Comunidades (BST + lista mensajes)

- Fase 1 (form_comunidades): crear comunidad + agregar miembros (root).
- Fase 2:
 - form_mensaje_comunidad: usuario publica mensaje en comunidades **a las que pertenece**.
 - FormMensajesComunidadesRoot: root ve **todas** las comunidades y sus mensajes.
 - Reporte: comunidades_bst.dot/png (muestra **nombre, fecha creación, #mensajes**).

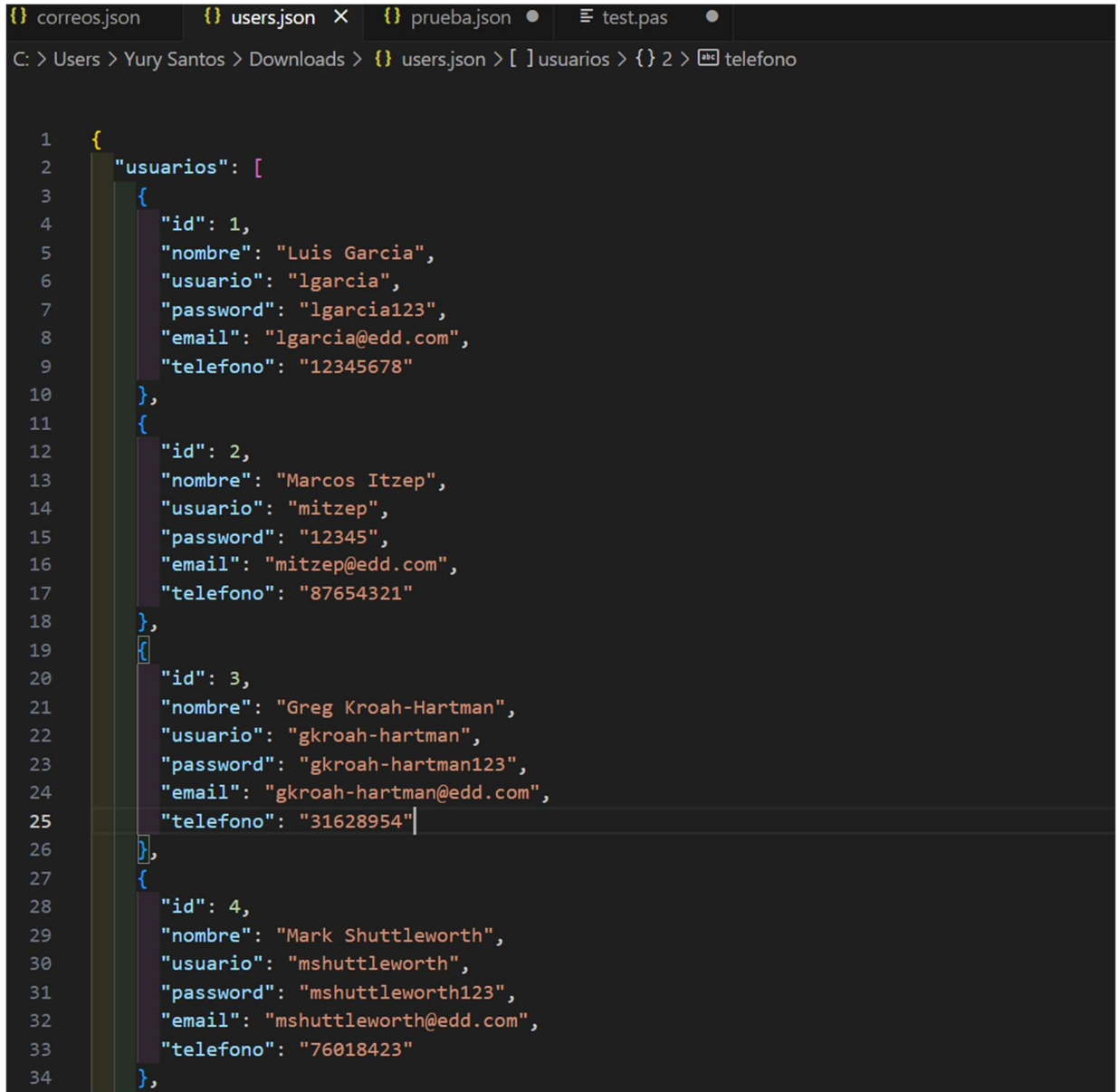
Carga masiva

- Usuarios: JSON con validación (IDs y emails únicos) → log de rechazados/aceptados.
- Correos: (opcional) carga a papelera según archivo correos.json (si se integró carga_masiva_correos.pas).

Persistencia / I/O

4.1 Archivos JSON

usuarios.json:



```
1 {
2   "usuarios": [
3     {
4       "id": 1,
5       "nombre": "Luis Garcia",
6       "usuario": "lgarcia",
7       "password": "lgarcia123",
8       "email": "lgarcia@edd.com",
9       "telefono": "12345678"
10    },
11    {
12      "id": 2,
13      "nombre": "Marcos Itzep",
14      "usuario": "mitzep",
15      "password": "12345",
16      "email": "mitzep@edd.com",
17      "telefono": "87654321"
18    },
19    {
20      "id": 3,
21      "nombre": "Greg Kroah-Hartman",
22      "usuario": "gkroah-hartman",
23      "password": "gkroah-hartman123",
24      "email": "gkroah-hartman@edd.com",
25      "telefono": "31628954"
26    },
27    {
28      "id": 4,
29      "nombre": "Mark Shuttleworth",
30      "usuario": "mshuttleworth",
31      "password": "mshuttleworth123",
32      "email": "mshuttleworth@edd.com",
33      "telefono": "76018423"
34    },
35  ]
36 }
```

- Reglas: **ID único** y **email único**. Rechazados se reportan en el log de carga masiva.

contactos.json (si se persisten contactos): listado por dueño/email.

correos.json (si se cargan masivamente correos a papelera):

```
{} correos.json X {} users.json {} prueba.json test.pas
C: > Users > Yury Santos > Desktop > New folder (5) > -EDD-2S2025_202113318-main > {} correos.json > [ ] correos > {} 10 > destino
1
2  {
3    "correos": [
4      {
5        "id": 1,
6        "remite": "lgarcia@edd.com",
7        "destinatario": "mitzep@edd.com",
8        "estado": "NL",
9        "asunto": "Reuni3n de proyecto",
10       "mensaje": "Confirmar asistencia a la reuni3n de equipo del lunes."
11     },
12     {
13       "id": 2,
14       "remite": "lgarcia@edd.com",
15       "destinatario": "mitzep@edd.com",
16       "estado": "LEIDO",
17       "asunto": "Reporte semanal",
18       "mensaje": "Adjunto el reporte semanal de actividades del 3rea de desarrollo."
19     },
20     {
21       "id": 3,
22       "remite": "mitzep@edd.com",
23       "destinatario": "gkroah-hartman@edd.com",
24       "estado": "NL",
25       "asunto": "Actualizaci3n de servidor",
26       "mensaje": "Se implementaron cambios importantes en el servidor. Por favor revisa los logs."
27     },
28     {
29       "id": 4,
30       "remite": "gkroah-hartman@edd.com",
31       "destinatario": "mshuttleworth@edd.com",
32       "estado": "LEIDO",
33       "asunto": "Nueva propuesta para el kernel",
34       "mensaje": "Te envi3 un borrador con la propuesta de mejora para revisi3n."
35     },
36     {
37       "id": 5,
38       "remite": "mshuttleworth@edd.com",
39       "destinatario": "smejia@edd.com",
40       "estado": "NL",
41       "asunto": "Invitaci3n a evento",
42       "mensaje": "Est3s cordialmente invitado al evento de innovaci3n tecnol3gica de este mes."
43     },
44     {
45       "id": 6,
46       "remite": "smejia@edd.com",
47       "destinatario": "bfox@edd.com",
48       "estado": "LEIDO",
49       "asunto": "Consulta sobre scripts",
50       "mensaje": "Tengo dudas con un script de automatizaci3n, ¿podr3as ayudarme?"
51     }
52   ]
53 }
```

Reportes (Graphviz)

- Carpeta destino: {usuario}-Reportes/ (ej. lgarcia-Reportes/).
- Archivos .dot y .png:
correos_recibidos, papelera, programados, contactos,
borradores_avl, contactos_bst, favoritos_btree, comunidades_bst.

Compilación y ejecución

Requisitos

- Lazarus/FreePascal (modo ObjFPC).
- **Graphviz** instalado para generar PNG:
 - Linux (Debian/Ubuntu): `sudo apt install graphviz`
 - Verifica binario: `/usr/bin/dot` o `which dot`.

Ejecución

- Ejecutable en el directorio del proyecto.
- Mantener los JSON (usuarios.json, contactos.json, etc.) junto al binario o usar rutas absolutas.